Classification Metrics:

A confusion matrix is a table that is often used to **describe the performance of a classification model** (or "classifier") on a set of test data for which the true values are known.

Let's start with an **example confusion matrix for a binary classifier** (though it can easily be extended to the case of more than two classes):

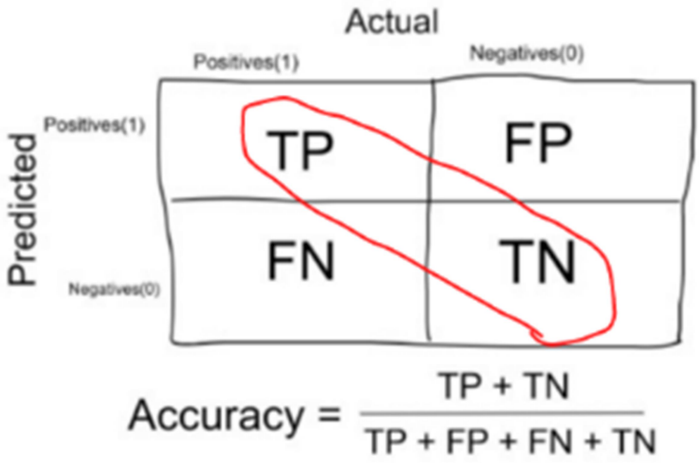
n=165	**Predicted: NO**	**Predicted: YES**
Actual: NO	50	10
Actual: YES	5	100

I've added these terms to the **confusion matrix**, and also added the row and column totals:

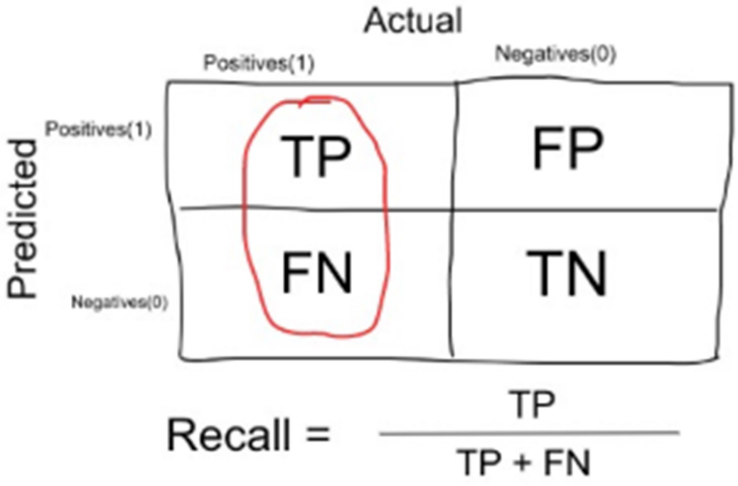
n=165	**Predicted: NO**	**Predicted: YES**	
Actual: NO	TN = 50	FP = 10	60
Actual: YES	FN = 5	TP = 100	105
	55	110	

This is a list of rates that are often computed from a confusion matrix for a binary classifier:

- **Accuracy:** Overall, how often is the classifier correct?

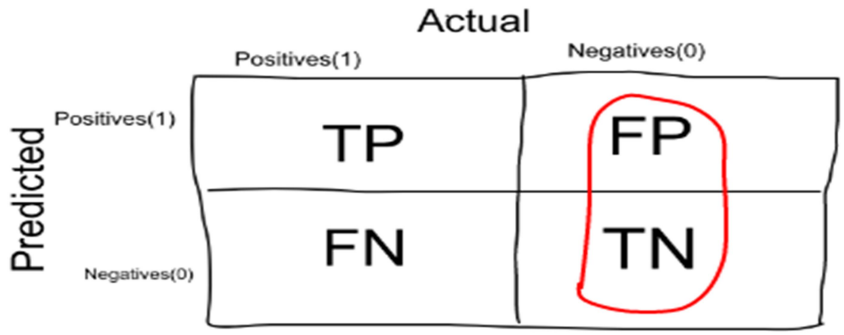


  - (TP+TN)/total = (100+50)/165 = 0.91
- **Misclassification Rate:** Overall, how often is it wrong?
  - (FP+FN)/total = (10+5)/165 = 0.09
  - equivalent to 1 minus Accuracy
  - also known as "Error Rate"
- **True Positive Rate:** When it's actually yes, how often does it predict yes?



  - TP/actual yes = 100/105 = 0.95
  - also known as "Sensitivity" or "Recall"

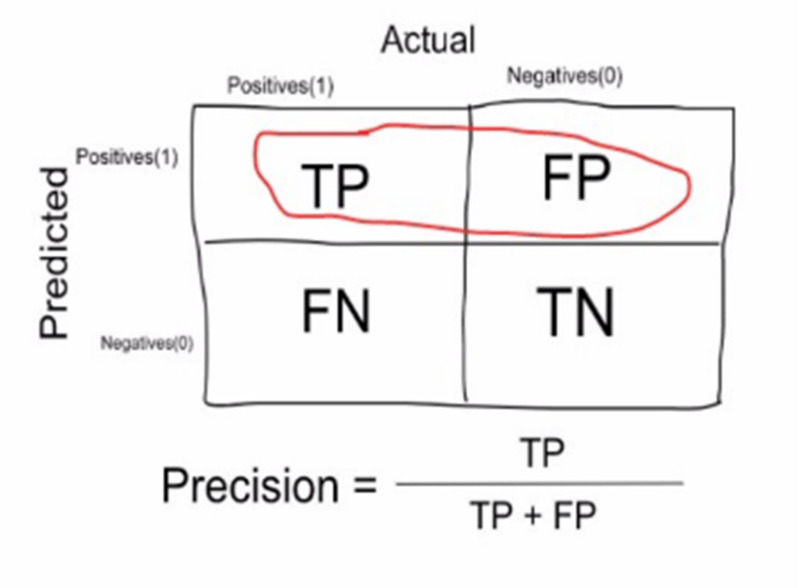
- **False Positive Rate:** When it's actually no, how often does it predict yes?
-
    - FP/actual no = 10/60 = 0.17
    -
- **True Negative Rate:** When it's actually no, how often does it predict no?



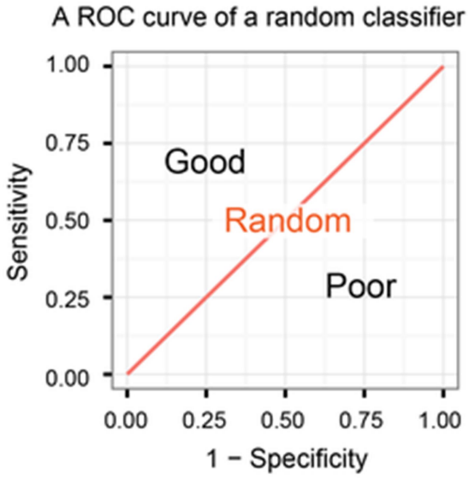
$$\text{Specificity} = \frac{TN}{TN + FP}$$

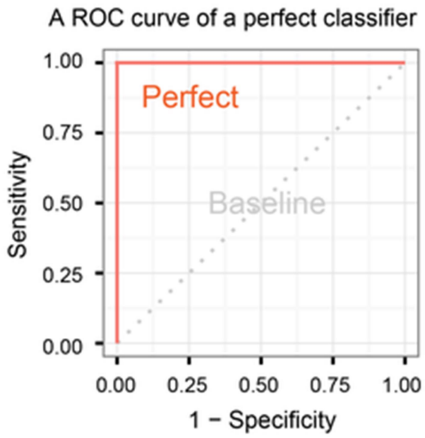
-
    - TN/actual no = 50/60 = 0.83
    - equivalent to 1 minus False Positive Rate
    - also known as "Specificity"

- **Precision:** When it predicts yes, how often is it correct?



-
  - TP/predicted yes = 100/110 = 0.91
  -
- **Prevalence:** How often does the yes condition actually occur in our sample?
  - actual yes/total = 105/165 = 0.64
  -
- **F Score:** This is a weighted average of the true positive rate (recall) and precision.
-
- F1 Score = 2 * Precision * Recall / (Precision + Recall)
-
- **ROC Curve:** This is a commonly used graph that summarizes the performance of a classifier over all possible thresholds. It is generated by plotting the True Positive Rate (y-axis) against the False Positive Rate (x-axis) as you vary the threshold for assigning observations to a given class.





Confusion Matrix using scikit-learn in Python

```
# importing all necessary libraries
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
from sklearn.metrics import plot_confusion_matrix

# confusion_matrix funnction a matrix containing the summary of predictions
print(confusion_matrix(y_test, predictions))

# plot_confusion_matrix function is used to visualize the confusion matrix
plot_confusion_matrix(classifier, X_test, y_test)
plt.show()
```

Precision:

```
    # Importing all necessary libraries
    from sklearn.metrics import precision_score

    # Calculating the precision score of classifier
    print(f"Precision Score of the classifier is: {precision_score(y_test, predictions)}")
```

Recall score

```
    # Importing all necessary libraries
    from sklearn.metrics import recall_score

    # Calculating the recall score of classifier
    print(f"Recall Score of the classifier is: {recall_score(y_test, predictions)}")
```

F1 score

```
    # Importing all necessary libraries
    from sklearn.metrics import f1_score

    # Calculating the F1 score of classifier
    print(f"F1 Score of the classifier is: {f1_score(y_test, predictions)}")
```

ROC curve

```
# Importing all necessary libraries
from sklearn.metrics import roc_curve, auc

class_probabilities = classifier.predict_proba(X_test)
preds = class_probabilities[:, 1]

fpr, tpr, threshold = roc_curve(y_test, preds)
roc_auc = auc(fpr, tpr)

# Printing AUC
print(f"AUC for our classifier is: {roc_auc}")

# Plotting the ROC
plt.title('Receiver Operating Characteristic')
plt.plot(fpr, tpr, 'b', label = 'AUC = %0.2f' % roc_auc)
plt.legend(loc = 'lower right')
plt.plot([0, 1], [0, 1],'r--')
plt.xlim([0, 1])
plt.ylim([0, 1])
plt.ylabel('True Positive Rate')
plt.xlabel('False Positive Rate')
plt.show()
```

Regression Metrics:

1. True Values and Predicted Values:

In regression, we've got two units of values to compare: the actual target values (authentic values) and the values expected by our version (anticipated values). The performance of the model is assessed by means of measuring the similarity among these sets.

2. Evaluation Metrics:

Regression metrics are quantitative measures used to evaluate the goodness of a regression model. Scikit-analyze provide's several metrics, each with its own strengths and boundaries, to assess how well a model suits the statistics.

Types of Regression Metrics:

Some common regression metrics in scikit-learn with examples.

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- R-squared (R^2) Score
- Root Mean Squared Error (RMSE)

Mean Absolute Error (MAE)

In the fields of statistics and machine learning, the Mean Absolute Error (MAE) is a frequently employed metric. It's a measurement of the typical absolute discrepancies between a dataset's actual values and predicted values.

Mathematical Formula

The formula to calculate MAE for a data with "n" data points is:

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |x_i - y_i|$$

Where:

- x_i represents the actual or observed values for the i-th data point.
- y_i represents the predicted value for the i-th data point.

Mean Squared Error (MSE):

A popular metric in statistics and machine learning is the Mean Squared Error (MSE). It measures the square root of the average discrepancies between a dataset's actual values and predicted values. MSE is frequently utilized in regression issues and is used to assess how well predictive models work.

Mathematical Formula

For a dataset containing 'n' data points, the MSE calculation formula is:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(x_i - y_i)^2$$

where:

- x_i represents the actual or observed value for the i-th data point.
- y_i represents the predicted value for the i-th data point.

R-squared (R^2) Score:

A statistical metric frequently used to assess the goodness of fit of a regression model is the R-squared (R_2) score, also referred to as the coefficient of determination. It quantifies the percentage of the dependent variable's variation that the model's independent variables contribute to. R_2 is a useful statistic for evaluating the overall effectiveness and explanatory power of a regression model.

Mathematical Formula

The formula to calculate the R-squared score is as follows:

$$R^2 = 1 - \frac{RSS}{TSS}$$

R^2 = coefficient of determination

RSS = sum of squares of residuals

TSS = total sum of squares

$$residual\ sum\ of\ squares = \sum_{i=1}^{n}(y_i - \hat{y_i})^2$$

Where,

y_i is the observed variable value

$\hat{y}$ is the value estimated by the regression line

$$\mathrm{TSS} = \sum_{i=1}^{n} (y_i - \bar{y})^2$$

TSS = total sum of squares

n = number of observations

y_i = value in a sample

$\bar{y}$ = mean value of a sample

Root Mean Squared Error (RMSE):

RMSE stands for Root Mean Squared Error. It is usually used metric in regression analysis and machine learning to measure the accuracy or goodness of fit of a predictive model, especially when the predictions are continuous numerical values.

The RMSE quantifies how well the predicted values from a model align with the actual observed values in the dataset. Here's how it works:

1. **Calculate the Squared Differences:** For each data point, subtract the predicted value from the actual (observed) value, square the result, and sum up these squared differences.
2. **Compute the Mean:** Divide the sum of squared differences by the number of data points to get the mean squared error (MSE).
3. **Take the Square Root:** To obtain the RMSE, simply take the square root of the MSE.

Mathematical Formula

The formula for RMSE for a data with 'n' data points is as follows:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - y_i)^2}$$

Where:

- RMSE is the Root Mean Squared Error.
- x_i represents the actual or observed value for the i-th data point.
- y_i represents the predicted value for the i-th data point.

Mean Squared Error

```
# example of calculate the mean squared error
from sklearn.metrics import mean_squared_error
# real value
expected = [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
# predicted value
predicted = [1.0, 0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2, 0.1, 0.0]
# calculate errors
errors = mean_squared_error(expected, predicted)
# report error
print(errors)
```

Root Mean Squared Error

```
# example of calculate the root mean squared error
from sklearn.metrics import mean_squared_error
# real value
expected = [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
# predicted value
predicted = [1.0, 0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2, 0.1, 0.0]
# calculate errors
errors = mean_squared_error(expected, predicted, squared=False)
# report error
print(errors)
```

Mean Absolute Error:

```
# example of calculate the mean absolute error
from sklearn.metrics import mean_absolute_error
# real value
expected = [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
# predicted value
predicted = [1.0, 0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2, 0.1, 0.0]
# calculate errors
errors = mean_absolute_error(expected, predicted)
# report error
print(errors)
```